DSPP

Running Code

```
# library(magick)
# library(image.libfacedetection)
#
# # setwd("C:/path/to/your/WIDER_FACE_folder")
#
# calculate_detection_rate <- function(folder_path, confidence_t
#   image_files <- list.files(folder_path, full.names = TRUE)
#   image_files <- image_files[!is.na(image_files)]
#
#   if(length(image_files) == 0) return(0)
#
#   # Randomly sample up to 100 images to prevent crashing while
#   actual_sample_size <- min(sample_size, length(image_files))
#   sampled_files <- sample(image_files, actual_sample_size)
#
#   success_count <- 0
#
#   for (img_path in sampled_files) {
#     img <- tryCatch(image_read(img_path), error = function(e)
#
#     if (!is.null(img)) {
#       faces <- image_detect_faces(img)
#       if (length(faces$detections) > 0 && any(faces$detections
#         success_count <- success_count + 1
#       }
#     }
#   }
#   return((success_count / actual_sample_size) * 100)
# }
```

```
# # Compare 3 tiers of Scale
# rate_parade <-
calculate_detection_rate("WIDER_train/images/0--Parade/") #
Extreme Crowd
# rate_marching <-
calculate_detection_rate("WIDER_train/images/10--
People_Marching/") # Medium Crowd
# rate_students <-
calculate_detection_rate("WIDER_train/images/31--
Student_Schoolkids/") # Close-up
#
```

```
# barplot(c(rate_parade, rate_marching, rate_students),
#         names.arg = c("Parade (Extreme)", "Marching
(Medium)", "Students (Close-up)"),
#         col = c("darkred", "orange", "darkgreen"),
#         main = "Detection Success by Subject Scale", ylab =
"Success Rate (%)")
```

```
# # Compare 3 tiers of Occlusion
# rate_riot <- calculate_detection_rate("WIDER_train/images/12--
# rate_racing <- calculate_detection_rate("WIDER_train/images/4
# rate_interview <- calculate_detection_rate("WIDER_train/image
#
# barplot(c(rate_riot, rate_racing, rate_interview),
#         names.arg = c("Riot (Heavy)", "Racing (Partial)", "In
#         col = c("darkred", "orange", "darkgreen"),
#         main = "Detection Success by Occlusion Level", ylab =
```

```
# # Compare 3 tiers of Pose
# rate_gymnastics <-
calculate_detection_rate("WIDER_train/images/17--Ceremony/") #
Highly Dynamic/Atypical
# rate_cop <- calculate_detection_rate("WIDER_train/images/14--
Traffic_Cop/") # Mixed Profiles
# rate_meeting <-
calculate_detection_rate("WIDER_train/images/11--Meeting/") #
Static/Frontal
#
# barplot(c(rate_gymnastics, rate_cop, rate_meeting),
#         names.arg = c("Ceremony (Atypical)", "Traffic Cop
(Mixed)", "Meeting (Frontal)"),
#         col = c("darkred", "orange", "darkgreen"),
#         main = "Detection Success by Pose & Dynamics", ylab =
"Success Rate (%)")
```

```
library(magick)
```

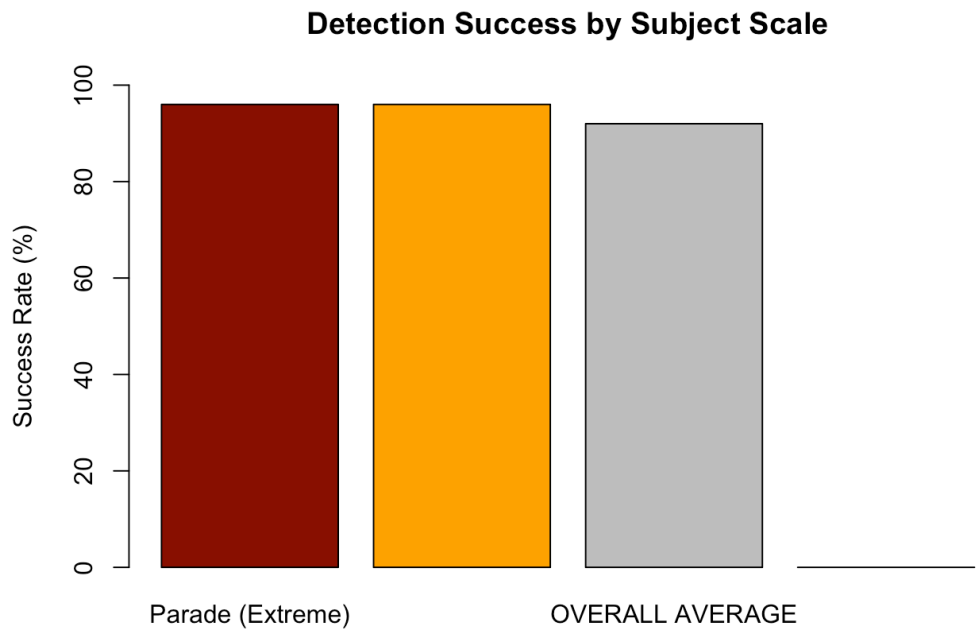
```
Linking to ImageMagick 6.9.13.29
Enabled features: cairo, fontconfig, freetype, heic, lcms,
pango, raw, rsvg, webp
Disabled features: fftw, ghostscript, x11
```

```
library(image.libfacedetection)

# setwd("C:/path/to/your/WIDER_FACE_folder")

calculate_detection_rate <- function(folder_path = NULL, is_glo
```

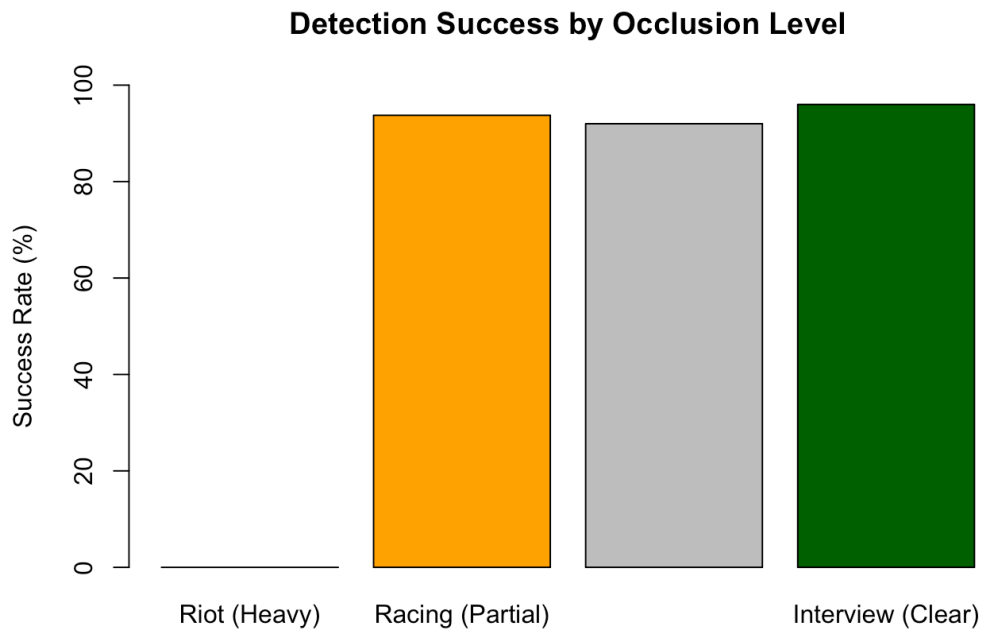
```
  # If calculating the global baseline, pull from all folders
  if(is_global) {
    image_files <- list.files("WIDER_train/images/", recursive =
  } else {
    image_files <- list.files(folder_path, full.names = TRUE, pa
  }

  image_files <- image_files[!is.na(image_files)]
  if(length(image_files) == 0) return(0)

  # Randomly sample exactly 100 images to prevent crashing
  actual_sample_size <- min(sample_size, length(image_files))
  sampled_files <- sample(image_files, actual_sample_size)

  success_count <- 0

  for (img_path in sampled_files) {
    img <- tryCatch(image_read(img_path), error = function(e) NU
    if (!is.null(img)) {
      faces <- image_detect_faces(img)
      if (length(faces$detections) > 0 && any(faces$detections$
        success_count <- success_count + 1
      }
    }
  }
  return((success_count / actual_sample_size) * 100)
}

# EVERYONE CALCULATES THE OVERALL BASELINE
rate_overall_baseline <- calculate_detection_rate(is_global = TR
print(paste("Overall Dataset Baseline:", rate_overall_baseline,
```

```
[1] "Overall Dataset Baseline: 92 %"
```

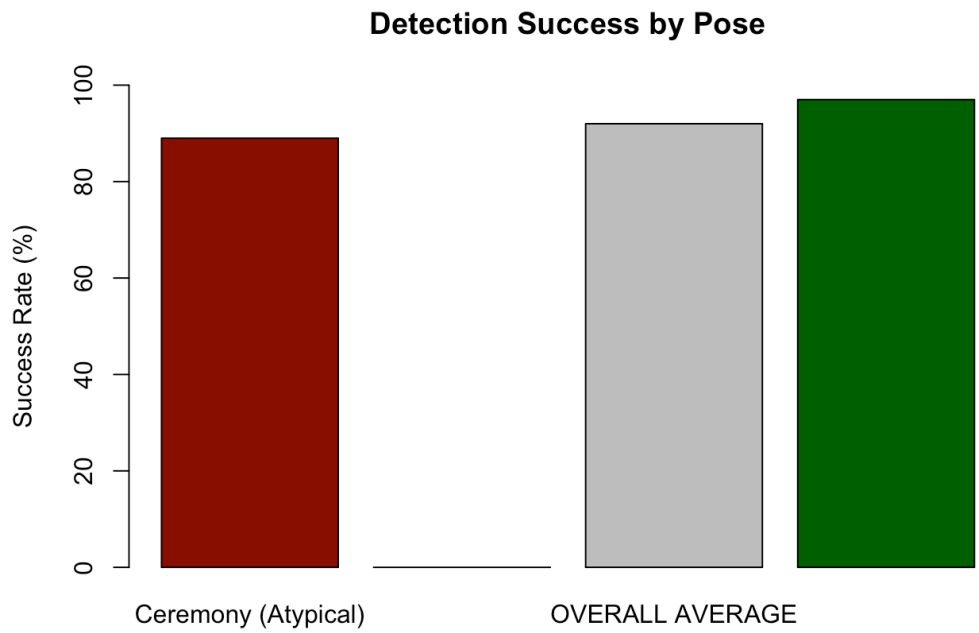
```
# Compare 3 tiers of Scale against the Baseline
rate_parade <- calculate_detection_rate("WIDER_train/images/0--P
rate_marching <- calculate_detection_rate("WIDER_train/images/10
rate_students <- calculate_detection_rate("WIDER_train/images/31

# Plot including the Overall Baseline
barplot(c(rate_parade, rate_marching, rate_overall_baseline, rat
        names.arg = c("Parade (Extreme)", "Marching (Medium)", "
        col = c("darkred", "orange", "gray", "darkgreen"),
        main = "Detection Success by Subject Scale", ylab = "Suc
```




```
# Compare 3 tiers of Occlusion against the Baseline
rate_riot <- calculate_detection_rate("WIDER_train/images/12--R
rate_racing <- calculate_detection_rate("WIDER_train/images/42--
rate_interview <- calculate_detection_rate("WIDER_train/images/

# Plot including the Overall Baseline
barplot(c(rate_riot, rate_racing, rate_overall_baseline, rate_i
        names.arg = c("Riot (Heavy)", "Racing (Partial)", "OVER
        col = c("darkred", "orange", "gray", "darkgreen"),
        main = "Detection Success by Occlusion Level", ylab = "
```




```
# Compare 3 tiers of Pose against the Baseline
rate_gymnastics <- calculate_detection_rate("WIDER_train/images/
rate_cop <- calculate_detection_rate("WIDER_train/images/14--Tra
rate_meeting <- calculate_detection_rate("WIDER_train/images/11-

# Plot including the Overall Baseline
barplot(c(rate_gymnastics, rate_cop, rate_overall_baseline, rate
        names.arg = c("Ceremony (Atypical)", "Traffic Cop (Mixed
        col = c("darkred", "orange", "gray", "darkgreen"),
        main = "Detection Success by Pose", ylab = "Success Rate
```




```r
folder_path <- "~/Documents/Spring 2026 Courses/DS for Public Po
filenames <- list.files(path = folder_path)
print(filenames)
```

```
 [1] "0--Parade"                    "1--Handshaking"
 [3] "10--People_Marching"          "11--Meeting"
 [5] "12--Group"                    "13--Interview"
 [7] "14--Traffic"                  "15--Stock_Market"
 [9] "16--Award_Ceremony"           "17--Ceremony"
[11] "18--Concerts"                 "19--Couple"
[13] "2--Demonstration"             "20--Family_Group"
[15] "21--Festival"                 "22--Picnic"
[17] "23--Shoppers"                 "24--Soldier_Firing"
[19] "25--Soldier_Patrol"           "26--Soldier_Drilling"
[21] "27--Spa"                      "28--Sports_Fan"
[23] "29--Students_Schoolkids"      "3--Riot"
[25] "30--Surgeons"                 "31--Waiter_Waitress"
[27] "32--Worker_Laborer"           "33--Running"
[29] "34--Baseball"                 "35--Basketball"
[31] "36--Football"                 "37--Soccer"
[33] "38--Tennis"                   "39--Ice_Skating"
[35] "4--Dancing"                   "40--Gymnastics"
[37] "41--Swimming"                 "42--Car_Racing"
[39] "43--Row_Boat"                 "44--Aerobics"
[41] "45--Balloonist"               "46--Jockey"
[43] "47--Matador_Bullfighter"      "48--
```

```
Parachutist_Paratrooper"
[45] "49--Greeting"                "5--Car_Accident"
[47] "50--Celebration_Or_Party"    "51--Dresses"
[49] "52--Photographers"           "53--Raid"
[51] "54--Rescue"                  "55--Sports_Coach_Trainer"
[53] "56--Voter"                   "57--Angler"
[55] "58--Hockey"                  "59--people--driving--car"
[57] "6--Funeral"                  "61--Street_Battle"
[59] "7--Cheering"                 "8--Election_Campain"
[61] "9--Press_Conference"
```

```r
list.dirs("WIDER_train/images/", recursive = FALSE)
```

```
 [1] "WIDER_train/images//0--Parade"
 [2] "WIDER_train/images//1--Handshaking"
 [3] "WIDER_train/images//10--People_Marching"
 [4] "WIDER_train/images//11--Meeting"
 [5] "WIDER_train/images//12--Group"
 [6] "WIDER_train/images//13--Interview"
 [7] "WIDER_train/images//14--Traffic"
 [8] "WIDER_train/images//15--Stock_Market"
 [9] "WIDER_train/images//16--Award_Ceremony"
[10] "WIDER_train/images//17--Ceremony"
[11] "WIDER_train/images//18--Concerts"
[12] "WIDER_train/images//19--Couple"
[13] "WIDER_train/images//2--Demonstration"
[14] "WIDER_train/images//20--Family_Group"
[15] "WIDER_train/images//21--Festival"
[16] "WIDER_train/images//22--Picnic"
[17] "WIDER_train/images//23--Shoppers"
[18] "WIDER_train/images//24--Soldier_Firing"
[19] "WIDER_train/images//25--Soldier_Patrol"
[20] "WIDER_train/images//26--Soldier_Drilling"
[21] "WIDER_train/images//27--Spa"
[22] "WIDER_train/images//28--Sports_Fan"
[23] "WIDER_train/images//29--Students_Schoolkids"
[24] "WIDER_train/images//3--Riot"
[25] "WIDER_train/images//30--Surgeons"
[26] "WIDER_train/images//31--Waiter_Waitress"
[27] "WIDER_train/images//32--Worker_Laborer"
[28] "WIDER_train/images//33--Running"
[29] "WIDER_train/images//34--Baseball"
[30] "WIDER_train/images//35--Basketball"
[31] "WIDER_train/images//36--Football"
[32] "WIDER_train/images//37--Soccer"
[33] "WIDER_train/images//38--Tennis"
[34] "WIDER_train/images//39--Ice_Skating"
```

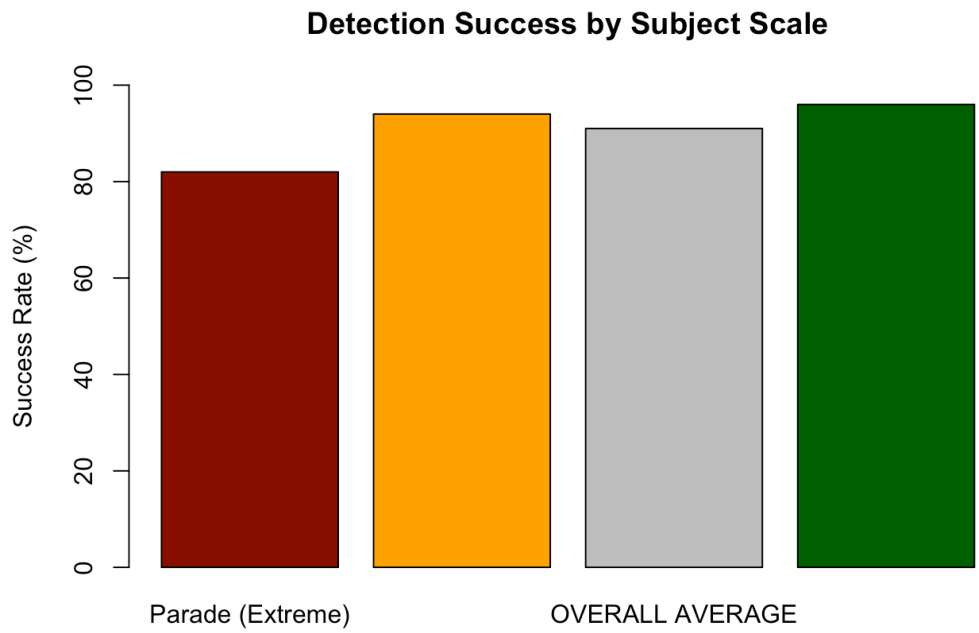
```
[35] "WIDER_train/images//4--Dancing"
[36] "WIDER_train/images//40--Gymnastics"
[37] "WIDER_train/images//41--Swimming"
[38] "WIDER_train/images//42--Car_Racing"
[39] "WIDER_train/images//43--Row_Boat"
[40] "WIDER_train/images//44--Aerobics"
[41] "WIDER_train/images//45--Balloonist"
[42] "WIDER_train/images//46--Jockey"
[43] "WIDER_train/images//47--Matador_Bullfighter"
[44] "WIDER_train/images//48--Parachutist_Paratrooper"
[45] "WIDER_train/images//49--Greeting"
[46] "WIDER_train/images//5--Car_Accident"
[47] "WIDER_train/images//50--Celebration_Or_Party"
[48] "WIDER_train/images//51--Dresses"
[49] "WIDER_train/images//52--Photographers"
[50] "WIDER_train/images//53--Raid"
[51] "WIDER_train/images//54--Rescue"
[52] "WIDER_train/images//55--Sports_Coach_Trainer"
[53] "WIDER_train/images//56--Voter"
[54] "WIDER_train/images//57--Angler"
[55] "WIDER_train/images//58--Hockey"
[56] "WIDER_train/images//59--people--driving--car"
[57] "WIDER_train/images//6--Funeral"
[58] "WIDER_train/images//61--Street_Battle"
[59] "WIDER_train/images//7--Cheering"
[60] "WIDER_train/images//8--Election_Campain"
[61] "WIDER_train/images//9--Press_Conference"
```

```
library(magick)
library(image.libfacedetection)

# setwd("C:/path/to/your/WIDER_FACE_folder")

calculate_detection_rate <- function(folder_path = NULL, is_glol

  if(is_global) {
    # If calculating the global baseline, pull from all folders
    image_files <- list.files("WIDER_train/images/", recursive =
  } else {
    # Pull from the specific folder path
    image_files <- list.files(folder_path, full.names = TRUE, pa
  }

  image_files <- image_files[!is.na(image_files)]
  if(length(image_files) == 0) return(0)

  # Randomly sample exactly 100 images to prevent crashing
```

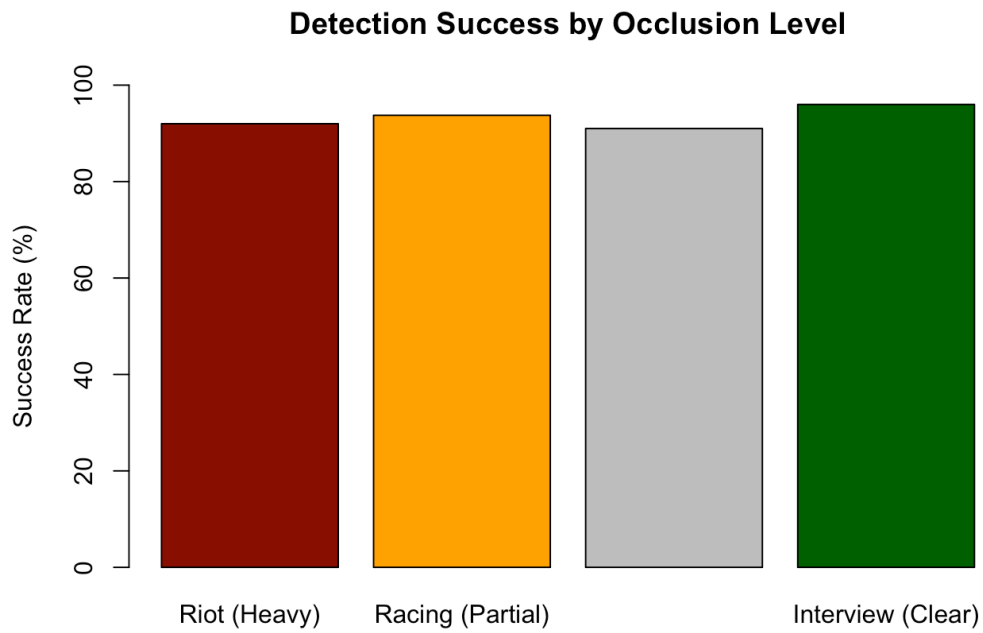
```
  actual_sample_size <- min(sample_size, length(image_files))
  sampled_files <- sample(image_files, actual_sample_size)

  success_count <- 0

  for (img_path in sampled_files) {
    img <- tryCatch(image_read(img_path), error = function(e) NU
    if (!is.null(img)) {
      faces <- image_detect_faces(img)
      if (length(faces$detections) > 0 && any(faces$detections$c
        success_count <- success_count + 1
      }
    }
  }
  return((success_count / actual_sample_size) * 100)
}

# EVERYONE CALCULATES THE OVERALL BASELINE
rate_overall_baseline <- calculate_detection_rate(is_global = TR
print(paste("Overall Dataset Baseline:", rate_overall_baseline,
```

```
[1] "Overall Dataset Baseline: 91 %"
```

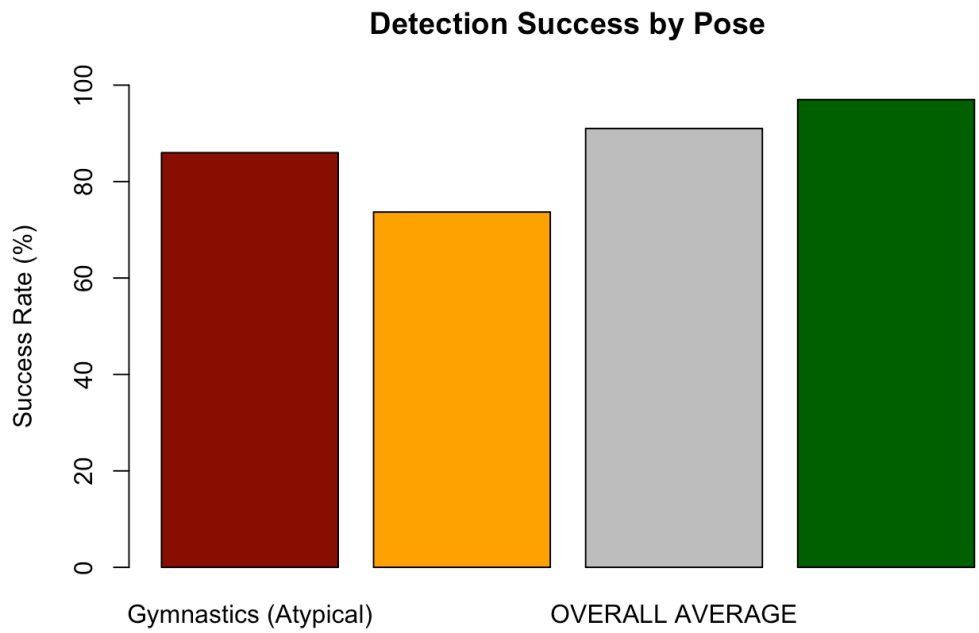
```
# Compare 3 tiers of Scale against the Baseline
rate_parade <- calculate_detection_rate("WIDER_train/images//0--
rate_marching <- calculate_detection_rate("WIDER_train/images//1
rate_students <- calculate_detection_rate("WIDER_train/images//2

# Plot
barplot(c(rate_parade, rate_marching, rate_overall_baseline, rat
        names.arg = c("Parade (Extreme)", "Marching (Medium)", "
        col = c("darkred", "orange", "gray", "darkgreen"),
        main = "Detection Success by Subject Scale", ylab = "Suc
```




```
# Compare 3 tiers of Occlusion against the Baseline
rate_riot <- calculate_detection_rate("WIDER_train/images//3--R
rate_racing <- calculate_detection_rate("WIDER_train/images//42
rate_interview <- calculate_detection_rate("WIDER_train/images/

# Plot
barplot(c(rate_riot, rate_racing, rate_overall_baseline, rate_i
        names.arg = c("Riot (Heavy)", "Racing (Partial)", "OVER
        col = c("darkred", "orange", "gray", "darkgreen"),
        main = "Detection Success by Occlusion Level", ylab = "
```




```
# Compare 3 tiers of Pose against the Baseline
rate_gymnastics <- calculate_detection_rate("WIDER_train/images/
rate_traffic <- calculate_detection_rate("WIDER_train/images//14
rate_meeting <- calculate_detection_rate("WIDER_train/images//11

# Plot
barplot(c(rate_gymnastics, rate_traffic, rate_overall_baseline,
        names.arg = c("Gymnastics (Atypical)", "Traffic (Mixed)"
        col = c("darkred", "orange", "gray", "darkgreen"),
        main = "Detection Success by Pose", ylab = "Success Rate
```

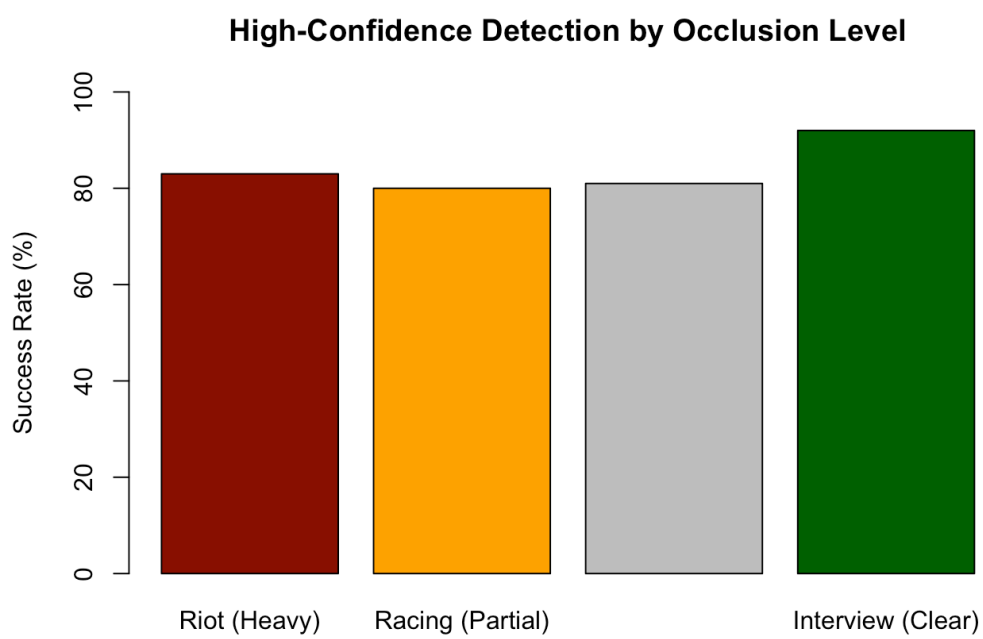



```
library(magick)
library(image.libfacedetection)

# UPDATED FUNCTION: Demanding 85% algorithmic confidence
calculate_detection_rate <- function(folder_path = NULL, is_glol

  if(is_global) {
    image_files <- list.files("WIDER_train/images/", recursive =
  } else {
    image_files <- list.files(folder_path, full.names = TRUE, pa
  }

  image_files <- image_files[!is.na(image_files)]
  if(length(image_files) == 0) return(0)

  actual_sample_size <- min(sample_size, length(image_files))
  sampled_files <- sample(image_files, actual_sample_size)

  success_count <- 0

  for (img_path in sampled_files) {
    img <- tryCatch(image_read(img_path), error = function(e) NU
    if (!is.null(img)) {
      faces <- image_detect_faces(img)

      # We now demand the AI is 85%+ confident before we count
      if (length(faces$detections) > 0 && any(faces$detections$c
```

```
        success_count <- success_count + 1
      }
    }
  }
  return((success_count / actual_sample_size) * 100)
}

# 1. Calculate the overall baseline with the strict 85% rule
rate_overall_baseline <- calculate_detection_rate(is_global = TI
print(paste("Overall Dataset Baseline (High Confidence):", rate_
```

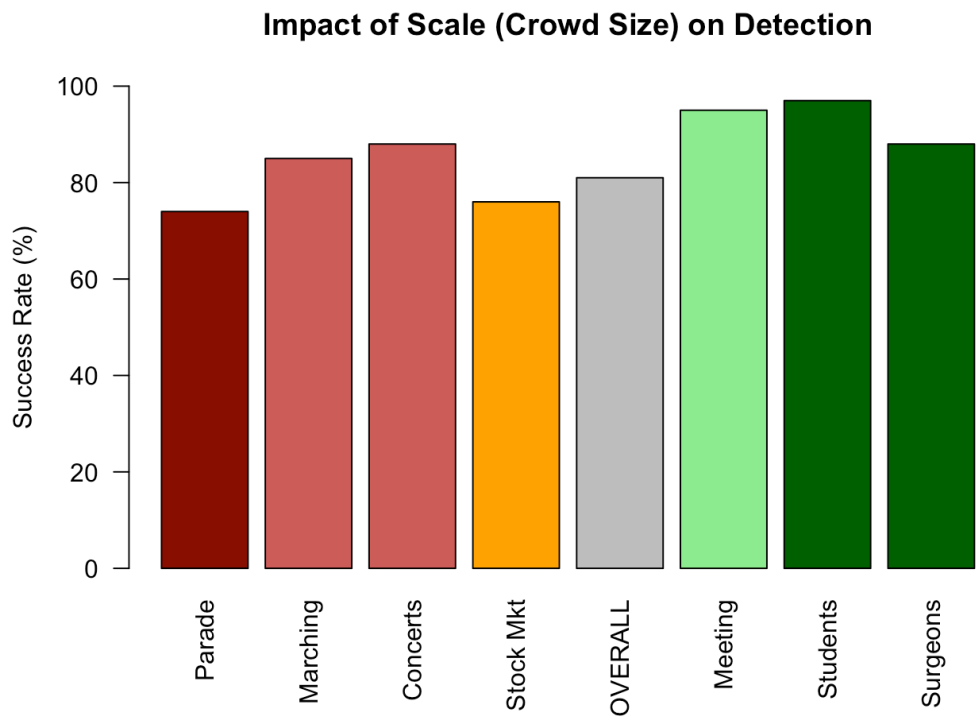
```
[1] "Overall Dataset Baseline (High Confidence): 81 %"
```

```
# 2. Re-run your Occlusion variables
rate_riot <- calculate_detection_rate("WIDER_train/images//3--R
rate_racing <- calculate_detection_rate("WIDER_train/images//42-
rate_interview <- calculate_detection_rate("WIDER_train/images/

# 3. Plot the new, accurate graph
barplot(c(rate_riot, rate_racing, rate_overall_baseline, rate_i
        names.arg = c("Riot (Heavy)", "Racing (Partial)", "OVER
        col = c("darkred", "orange", "gray", "darkgreen"),
        main = "High-Confidence Detection by Occlusion Level",
```



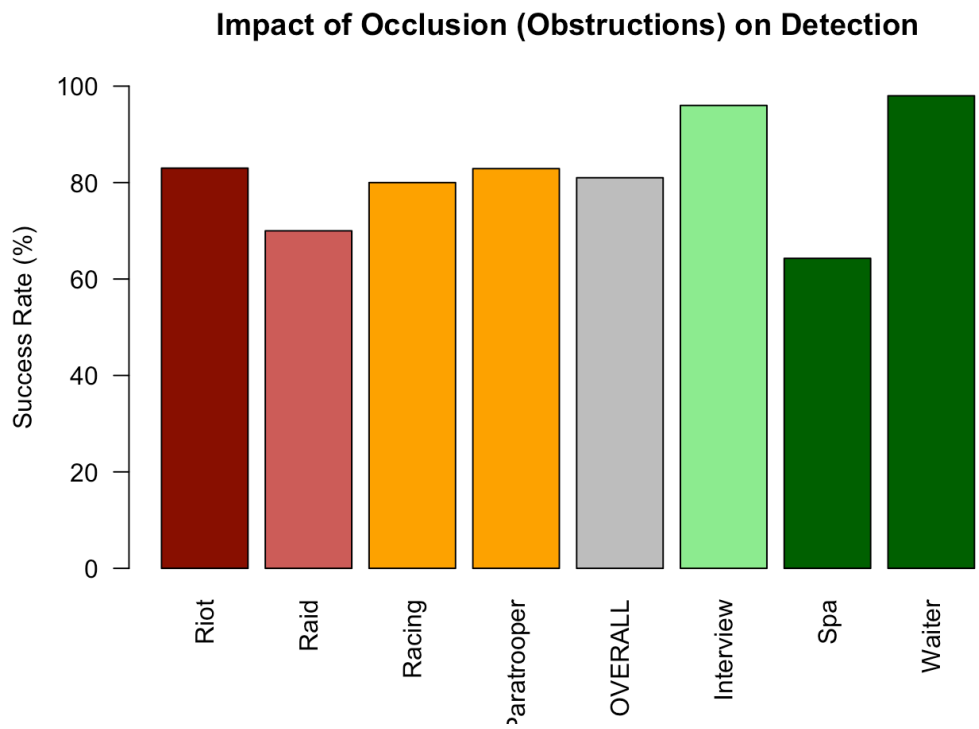

```
# 1. Run the folders
rate_parade <- calculate_detection_rate("WIDER_train/images//0--
```

```
rate_marching <- calculate_detection_rate("WIDER_train/images//1
rate_concerts <- calculate_detection_rate("WIDER_train/images//1
rate_stock <- calculate_detection_rate("WIDER_train/images//15--
rate_meeting <- calculate_detection_rate("WIDER_train/images//11
rate_students <- calculate_detection_rate("WIDER_train/images//2
rate_surgeons <- calculate_detection_rate("WIDER_train/images//1

# 2. Plot the 7 Bars + Overall Baseline
bars_scale <- c(rate_parade, rate_marching, rate_concerts, rate_
names_scale <- c("Parade", "Marching", "Concerts", "Stock Mkt",
colors_scale <- c("darkred", "indianred", "indianred", "orange",

barplot(bars_scale, names.arg = names_scale, col = colors_scale,
        main = "Impact of Scale (Crowd Size) on Detection", ylab
```

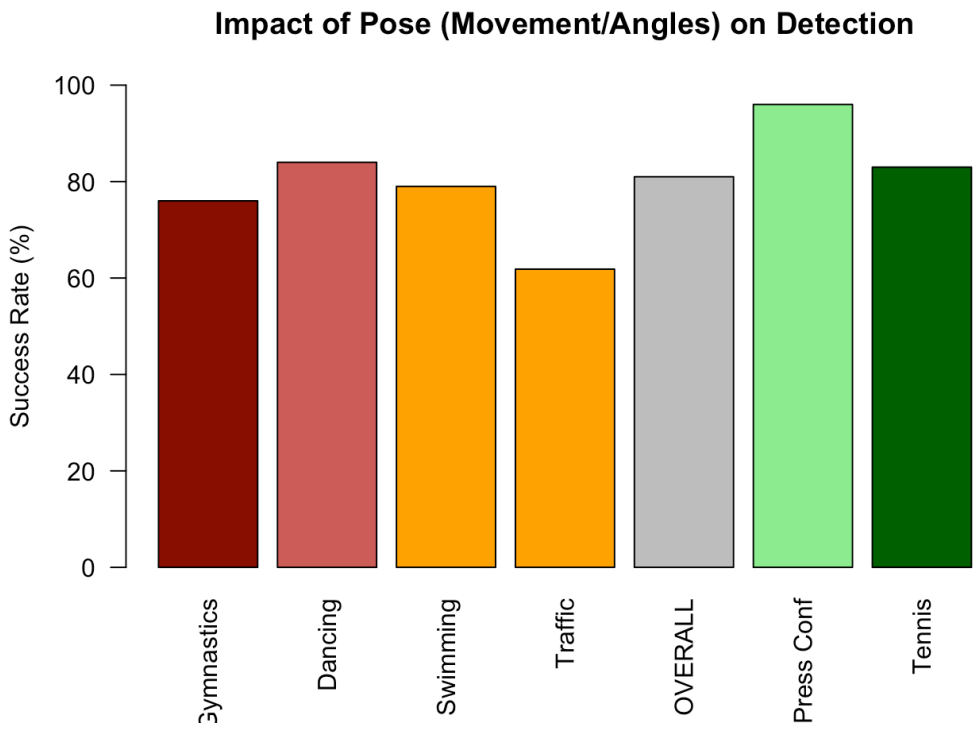



```
# 1. Run the folders
rate_riot <- calculate_detection_rate("WIDER_train/images//3--R
rate_raid <- calculate_detection_rate("WIDER_train/images//53--
rate_racing <- calculate_detection_rate("WIDER_train/images//42-
rate_paratrooper <- calculate_detection_rate("WIDER_train/images
rate_interview <- calculate_detection_rate("WIDER_train/images/
rate_spa <- calculate_detection_rate("WIDER_train/images//27--Sp
rate_waitress <- calculate_detection_rate("WIDER_train/images//1

# 2. Plot the 7 Bars + Overall Baseline
bars_occ <- c(rate_riot, rate_raid, rate_racing, rate_paratroope
```

```
names_occ <- c("Riot", "Raid", "Racing", "Paratrooper", "OVERALL
colors_occ <- c("darkred", "indianred", "orange", "orange", "gra

barplot(bars_occ, names.arg = names_occ, col = colors_occ, las =
        main = "Impact of Occlusion (Obstructions) on Detection"
```




```
# 1. Run the folders
rate_gymnastics <- calculate_detection_rate("WIDER_train/images/
rate_dancing <- calculate_detection_rate("WIDER_train/images//4-
rate_swimming <- calculate_detection_rate("WIDER_train/images//4
rate_traffic <- calculate_detection_rate("WIDER_train/images//14
rate_press <- calculate_detection_rate("WIDER_train/images//9--I
rate_tennis <- calculate_detection_rate("WIDER_train/images//38-

# 2. Plot the 6 Bars + Overall Baseline
bars_pose <- c(rate_gymnastics, rate_dancing, rate_swimming, rat
names_pose <- c("Gymnastics", "Dancing", "Swimming", "Traffic",
colors_pose <- c("darkred", "indianred", "orange", "orange", "gr

barplot(bars_pose, names.arg = names_pose, col = colors_pose, la
        main = "Impact of Pose (Movement/Angles) on Detection",
```


Impact of Pose (Movement/Angles) on Detection
Success Rate (%)
100
80
60
40
20
0
Gymnastics
Dancing
Swimming
Traffic
OVERALL
Press Conf
Tennis